INFORMATIONSBLATT

IB02 · Prometheus und Grafana – Professionelles Monitoring

LF10b · LS04 – Monitoring und Alerting einrichten · IB02 – Prometheus und Grafana | IT-3B · Schuljahr 2025/2026

1 · Monitoring-Architektur

1.1 | Komponenten im Überblick

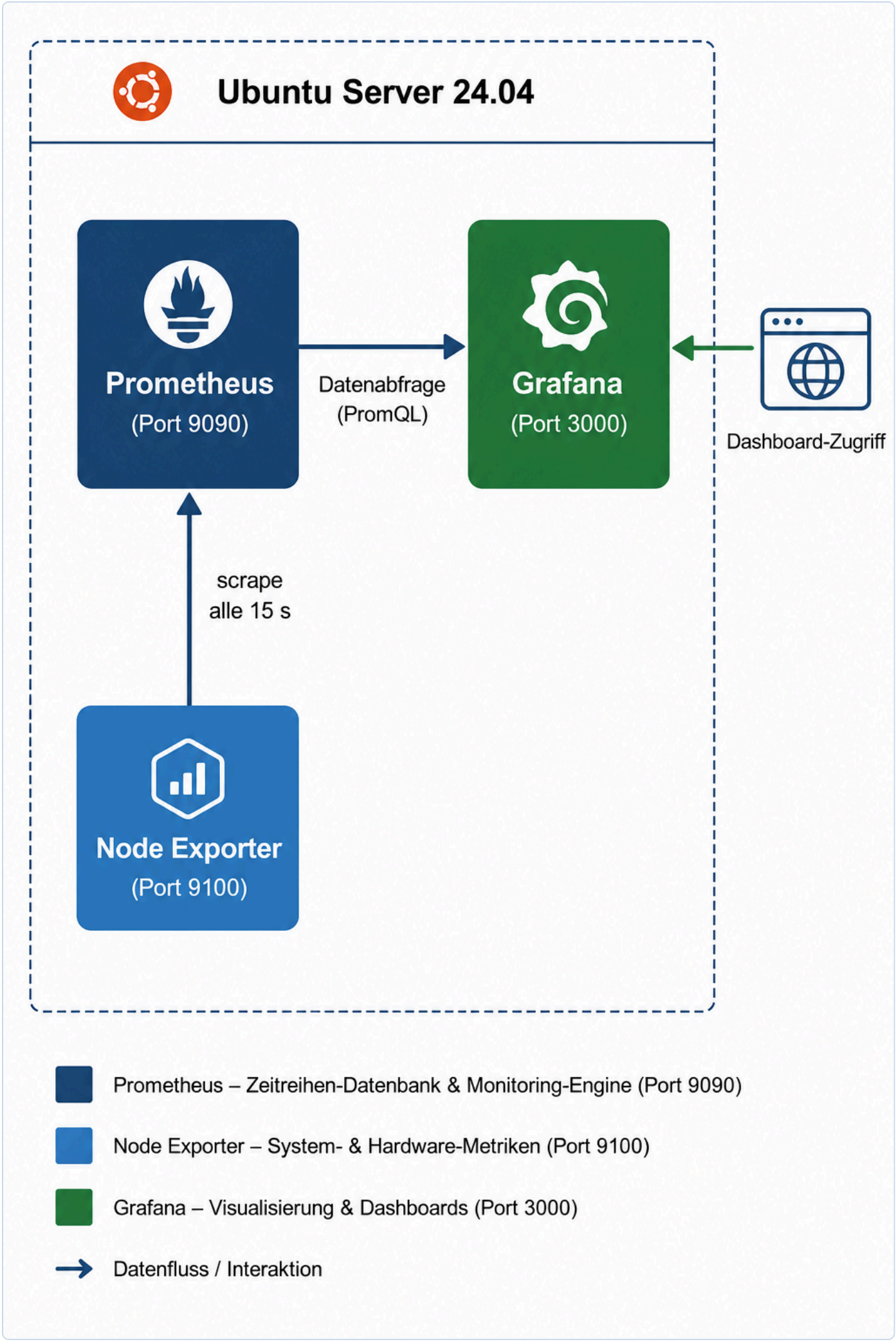

Ubuntu Server 24.04
Prometheus
(Port 9090)
Datenabfrage
(PromQL)
Grafana
(Port 3000)
Dashboard-Zugriff
scrape
alle 15 s
Node Exporter
(Port 9100)
Prometheus – Zeitreihen-Datenbank & Monitoring-Engine (Port 9090)
Node Exporter – System- & Hardware-Metriken (Port 9100)
Grafana – Visualisierung & Dashboards (Port 3000)
Datenfluss / Interaktion

Abbildung 1: Prometheus → Grafana Architekturdiagramm (Node Exporter scrape → Prometheus → Grafana-Dashboard)

Komponente	Aufgabe	Port	Paket (Ubuntu 24.04)
Node Exporter	Sammelt System-Metriken und stellt sie als HTTP-Endpunkt bereit	9100	`prometheus-node-exporter`
Prometheus	Ruft (scraped) Metriken von Targets ab und speichert sie als Zeitreihen	9090	`prometheus`
Grafana	Visualisiert Metriken aus Prometheus als interaktive Dashboards	3000	`grafana` (eigenes Repo)

1.2 | Das Pull-Prinzip

> 📖 **Pull-Prinzip vs. Push-Prinzip**
> Prometheus arbeitet nach dem **Pull-Prinzip**: Prometheus fragt (*scraped*) die überwachten Systeme in regelmäßigen Abständen aktiv ab. Das ist das Gegenteil des **Push-Prinzips**, bei dem die überwachten Systeme selbst Daten an einen zentralen Server schicken.

	Pull (Prometheus)	Push (z. B. Zabbix-Agent im Aktiv-Modus, Nagios NSCA)
Wer initiiert?	Prometheus fragt aktiv an	Überwachtes System sendet aktiv
Vorteil	Zentrale Kontrolle; einfache Konfiguration	Funktioniert durch Firewalls (ausgehend)
Nachteil	Targets müssen erreichbar sein	Jedes System muss konfiguriert werden
Einsatz	Server-Monitoring im LAN	Verteilte Umgebungen hinter Firewalls

2 · Node Exporter

2.1 | Wichtige Metriken

Der Node Exporter stellt hunderte von Metriken bereit. Die wichtigsten für den Alltag:

Metrik-Name	Bedeutung	Praxisnutzen
`node_cpu_seconds_total`	Gesamte CPU-Zeit nach Modus (idle, user, system, …)	CPU-Auslastung berechnen
`node_memory_MemAvailable_bytes`	Verfügbarer Arbeitsspeicher in Bytes	RAM-Engpässe frühzeitig erkennen
`node_memory_MemTotal_bytes`	Gesamter Arbeitsspeicher	Basis für RAM-%-Berechnung

`node_filesystem_avail_bytes`	Freier Speicherplatz je Dateisystem	Festplatten-Volllauf verhindern
`node_filesystem_size_bytes`	Gesamtgröße je Dateisystem	Basis für Disk-%-Berechnung
`node_network_receive_bytes_total`	Empfangene Datenmenge je Interface	Netzwerklast überwachen
`node_load1`	Systemlast der letzten 1 Minute	Überlastsituationen erkennen
`node_boot_time_seconds`	Zeitpunkt des letzten Systemstarts	Uptime berechnen

> 💡 **Merke: Metriken-Format**
> Prometheus-Metriken folgen einem einheitlichen Format:
> `metrik_name{label="wert"} zahlenwert zeitstempel`
> Beispiel: `node_memory_MemAvailable_bytes{} 1073741824 1700000000000` bedeutet: 1 GiB RAM verfügbar (1.073.741.824 Bytes = 1024³ Bytes).

3 · Prometheus

3.1 | Installation

Die Installation finden Sie in der Schritt-für-Schritt-Anleitung.

3.2 | Konfigurationsdatei prometheus.yml

Prometheus liest seine Konfiguration aus `/etc/prometheus/prometheus.yml`. Die wichtigste Sektion ist `scrape_configs` – hier werden die Targets (Endpunkte) definiert, von denen Metriken abgerufen werden.

```
/etc/prometheus/prometheus.yml – fertige Konfiguration für LS04

global:
  scrape_interval:     15s    # Alle 15 Sekunden Metriken abrufen
  evaluation_interval: 15s    # Alle 15 Sekunden Alerting-Regeln auswerten

scrape_configs:
  # Prometheus überwacht sich selbst
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  # Node Exporter – System-Metriken des Servers
  - job_name: 'node'
    static_configs:
      - targets: ['localhost:9100']
```

Konfigurations-Element	Bedeutung
`scrape_interval: 15s`	Prometheus fragt Targets alle 15 Sekunden ab
`job_name: 'node'`	Interner Name für diese Gruppe von Targets
`targets: ['localhost:9100']`	Adresse des Node Exporters (intern)

3.3 | Konfiguration prüfen und anwenden

Konfiguration validieren und Prometheus neu laden

```
# Syntaxprüfung (Fehlermeldung vor dem Neustart abfangen)
promtool check config /etc/prometheus/prometheus.yml

# Prometheus neu starten
sudo systemctl restart prometheus

# Im Web-UI: Targets-Status prüfen
# → http://192.168.10.50:9090/targets
# Erwarteter Status: UP für beide Targets
```

⚠️ **Immer zuerst promtool prüfen!**
Ein Syntaxfehler in `prometheus.yml` verhindert den Start von Prometheus. Führen Sie deshalb immer zuerst `promtool check config` aus. Erst bei der Ausgabe **„SUCCESS"** starten Sie Prometheus neu.

4 · Grafana

4.1 | Installation und Konfiguration

Installation und Konfiguration finden Sie in der Schritt-für-Schritt-Anleitung.

LF10b – Serverdienste bereitstellen und Administrationsaufgaben automatisieren
LS04 – Monitoring und Alerting einrichten · IB02 – Prometheus und Grafana
Schuljahr 2025/2026